

Recovering Linear Arrangements from Rooted-Tree Arrangements

Working manuscript for expert review
23 September 2026

Abstract

We give an explicit reduction from threshold Linear Arrangement to threshold Rooted Tree Arrangement that recovers a source answer from every valid target answer. A clique forces the source vertices onto an ancestor chain. One vertex per source edge records its length; any such vertex inserted into the chain incurs enough extra clique cost to pay for its apparent saving. The construction has quadratic graph size and a polynomial decoder. Exhaustive and solver-based checks cover small positive and negative instances. Garey and Johnson already attribute a reduction between these problems to Gavril; we do not establish whether the construction here coincides with his.

1. Introduction

A linear arrangement assigns graph vertices distinct positions on a line and charges each edge the distance between its endpoints. A rooted-tree arrangement uses ancestor-comparable vertices instead. The tree can branch, so using the source graph unchanged does not give a reverse implication: the four-vertex star costs three on a rooted star but at least four on a line.

Garey and Johnson list a transformation from Optimal Linear Arrangement to Rooted Tree Arrangement in GT45 [1], citing Gavril [2]. The listing does not specify the construction or a decoder. The upstream rule issue documents why the unchanged graph fails to support witness extraction [3]. We give a construction and prove an exact cost relation that decodes every target tree. The historical originality of the gadget remains unresolved because the cited Gavril proof was unavailable for comparison.

Our main result is a deterministic pair of polynomial-time algorithms F and D . For every legal source instance x and every valid target output y for $F(x)$, the decoded output $D(x, y)$ is valid for x . In the main case, the minimum target cost is the minimum source cost plus an explicit offset.

2. Problems and encodings

Let $G = (V, E)$ be a finite simple graph with $n = |V|$ and $m = |E|$. A linear arrangement π bijects V with the positions $1, \dots, n$ and has cost

$$c_{\pi(G)} = \sum_{\{u,v\} \in E} |\pi(u) - \pi(v)|.$$

The source output is any arrangement of cost at most the integer threshold k , or NO-SOLUTION exactly when none exists. The empty graph has the empty arrangement of cost zero.

A rooted-tree arrangement of a simple graph H is a rooted tree on its graph vertices. Each graph edge joins two vertices comparable in the ancestor order. Its cost $c_{T(H)}$ is the sum of their tree distances. The target output is any such tree of cost at most threshold K , or NO-SOLUTION exactly when none exists. Naming a tree vertex by its corresponding graph vertex records the required bijection without loss of generality.

The executable input lists all vertices, including isolated ones, and all edges. Vertex labels are $0, \dots, n - 1$. The vertex count field must match this list. Thresholds are arbitrary finite integers in JSON decimal notation, whose length is within a constant factor of binary bit length. A target tree is a parent array with one root entry -1 . These details fix the domain on which the algorithms are total. Write $b(k)$ for the bit length of the encoded threshold.

3. Construction

For $n \geq 2$ and $k \geq 0$, make the original vertices a clique in H . For each source edge $e = \{u, v\}$, add a new vertex g_e adjacent exactly to u and v . There are no edges between gadget vertices. Define

$$C_n = \sum_{1 \leq i < j \leq n} (j - i) = \frac{n(n^2 - 1)}{6},$$

and set $K = C_n + k + 2m$. Figure 1 shows the construction on a three-vertex path.

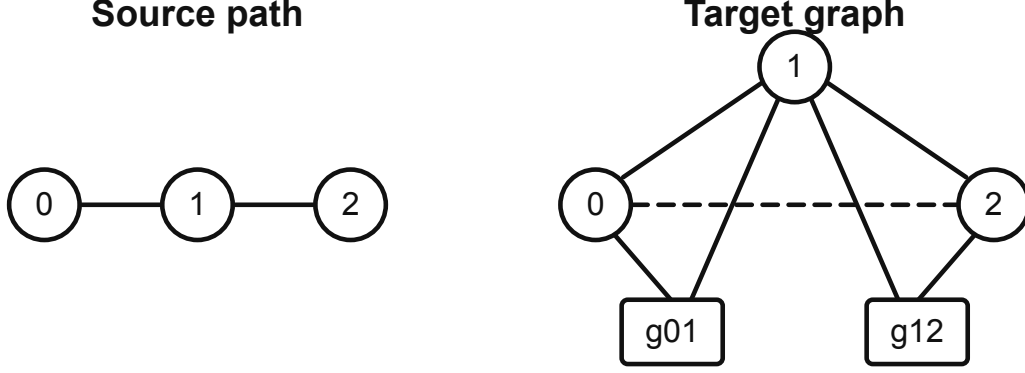


Figure 1: Construction for source edges $\{0, 1\}$ and $\{1, 2\}$. The original vertices are circles; each square g_e joins exactly the endpoints of its source edge. The dashed edge $\{0, 2\}$ completes the original-vertex clique. All seven target edges are shown.

If $k < 0$, output a triangle with $K = 1$. If $k \geq 0$ and $n \leq 1$, output one isolated vertex with $K = 1$. These cases give a positive target threshold even under the classical GT45 convention. They also cover the empty source graph.

Given a target tree witness in the main case, D follows parent pointers to find the depths of original vertices and lists them in increasing depth. For $n \leq 1$ it returns the empty or singleton order. Given target NO-SOLUTION, it returns source NO-SOLUTION. The next section proves every such response valid; D does not solve a new arrangement instance.

4. Cost identity and recovery

We first prove the upper bound. Fix any source order π . Put original vertices on a root-to-leaf path in that order. Attach each g_e as a child of the later endpoint of e . The clique edges cost C_n . If $e = \{u, v\}$ has rank difference $d_e = |\pi(u) - \pi(v)|$, its two gadget edges cost $d_e + 2$. Thus this tree costs $C_n + c_{\pi(G)} + 2m$.

For the lower bound, take an arbitrary valid target tree T . Every pair of original vertices is joined by a clique edge, so the originals form one ancestor chain. Write π_T for their order by depth. Let Q be the set of gadget vertices strictly between the first and last originals on this chain. For $g \in Q$, let a_g originals lie above it. Each original pair straddling g pays one extra unit of clique distance. Therefore the clique cost is

$$C_n + \sum_{g \in Q} a_g(n - a_g),$$

and each summand is at least one.

Now fix $e = \{u, v\}$ with u above v in T , and let d_e be their rank difference in π_T . Their tree distance D_e satisfies $D_e \geq d_e$. Comparability of g_e with both endpoints places it on the path from u to v , above u , or below v . Let $I_e = 1$ if g_e lies strictly between u and v , and $I_e = 0$ otherwise. In the first case its two incident edges cost $D_e \geq d_e + 1$. In the other cases they cost $D_e + 2t \geq d_e + 2$ for some integer $t \geq 1$. Hence the gadget-edge cost is at least $d_e + 2 - I_e$.

Every gadget counted by $I_e = 1$ belongs to Q and adds at least one unit to the clique cost. Combining the two charges gives

$$c_{T(H)} \geq C_n + \sum_e I_e + \sum_e (d_e + 2 - I_e) = C_n + c_{\pi_T}(G) + 2m.$$

Together with the upper bound, this proves our main identity:

$$\min_T c_{T(H)} = C_n + \min_{\pi} c_{\pi(G)} + 2m.$$

If a target witness costs at most K , the lower bound gives $c_{\pi_T}(G) \leq k$, so the decoder's order is a valid source output. The identity also makes target NO-SOLUTION equivalent to source NO-SOLUTION in the main case. When $k < 0$, source costs are nonnegative and the target triangle has minimum tree cost four, so both answer NO-SOLUTION. When $n \leq 1$ and $k \geq 0$, both have witnesses. This covers every legal source input and every valid target output, including alternative branching trees.

5. Complexity and scope

The main target has $n + m$ vertices and $\binom{n}{2} + 2m$ edges. The source encoding lists all n vertices, so n is at most input length even when E is empty. Since $m \leq \binom{n}{2}$, F takes polynomial time and writes $O(n^2 \log n + b(k))$ bits. Its threshold arithmetic has $O(\log(n + 1) + b(k))$ bits. The decoder traverses at most $n + m$ parent arcs for each original vertex and sorts the resulting depths, using $O(n(n + m) + n \log n)$ elementary operations on polynomial-size integers. The CLI removes Python's default decimal-integer digit cap before parsing or writing a legal threshold.

The result supplies the complete rule requested by the fixed question. It also yields the known NP-hardness consequence from the classical hardness of Linear Arrangement [1]. The exact construction's historical novelty is open pending inspection of Gavril's primary proof. The quadratic graph growth creates dense target instances, and our finite experiments make no claim about practical solver performance.

References

- [1] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*, W. H. Freeman, 1979, Appendix A1.3, entries GT42 and GT45. Online scan.
- [2] F. Gavril, "Some NP-complete problems on graphs," *Proceedings of the 11th Conference on Information Sciences and Systems*, Johns Hopkins University, 1977, pp. 91–95. Bibliographic attribution in [1]; original proof not inspected.
- [3] CodingThrust, "[Rule] OPTIMAL LINEAR ARRANGEMENT to ROOTED TREE ARRANGEMENT," problem-reductions issue 888, 2026. Issue page.

A. Verification and reproducibility

The proof above is the general correctness evidence. The prepared suite holds 114 fixed source cases: 100 seeded random and 14 edge cases, including 76 YES and 38 NO instances, through five source vertices and fifteen target vertices. It checks 188 target outputs, including 74 pairs of distinct valid trees. A separate verifier enumerates target trees for all simple source graphs through three vertices and all four-vertex graphs with at most two edges. At thresholds one below, at, and one above each source optimum, it checks 102 instances and 166 target outputs, including 34 NO-SOLUTION answers and 64 alternate tree pairs. The source optima come from independent exhaustive permutations; the larger prepared target instances use an exact component subset DP, and the small target oracle is cross-checked against Z3 and direct enumeration. These are finite checks, not a substitute for the argument above.

The recorded environment used Python 3.12.14, uv 0.12.17, Z3 5.1.0 (z3-solver 5.1.0.0), and Typst 0.15.1. Kissat 4.0.4 and Lean 4.34.0 with Lake 5.0.0 were available but unused. CP-SAT and Mathlib were absent and are not prerequisites. The technical-writing skill was available for manuscript preparation. From the repository root, reproduce the checks with:

```
w=campaigns/linear-arrangement-rooted-tree-arrangement/work
uv sync --locked
uv run --locked python research/validate_preparation.py "$w/cases.json"
uv run --locked python "$w/check.py" --self-test
uv run --locked python "$w/check.py" --candidate "$w/algorithm.py"
uv run --locked python "$w/verify.py" --candidate "$w/algorithm.py"
uv run --locked python "$w/evidence/large-integers/check.py"
```

The last check exercises 4,300- and 4,301-digit thresholds through both candidate modes. To run the maps directly:

```
w=campaigns/linear-arrangement-rooted-tree-arrangement/work
printf '%s\n' '{"n":2,"vertices":[0,1],"edges":[[0,1]],"k":1}' |
uv run --locked python "$w/algorithm.py"
printf '%s\n' '{"source":{"n":2,"vertices":[0,1],"edges":
[[0,1]],"k":1},"target_solution":{"parent":[-1,0,1]}}' |
uv run --locked python "$w/algorithm.py" --extract
```

The first command returns a triangle with threshold four; the second returns the source order [0, 1]. The fixed corpus, generators, candidate, oracle, proof, reviews and exact commands reside in this repository. No source or target solver timeout is used.